

Circuit parameter extraction

1 Generator vs GNN

The generator is a conductance-based network of N neurons. For neuron i ,

$$\tau_i \frac{dv_i}{dt} = -(v_i - V_i^{\text{rest}}) + \text{msg}_i + I_i, \quad \text{msg}_i = \sum_{j \in \mathcal{N}(i)} W_{ij} \text{relu}(v_j) (E_{ij} - v_i), \quad (1)$$

with $W_{ij} \geq 0$ a synaptic conductance in units of the leak conductance, and E_{ij} the reversal potential of the channel the presynaptic cell opens. The sign of a synapse lives entirely in the driving force $E_{ij} - v_i$, never in W_{ij} .

The model is a graph network with the same connectivity:

$$\frac{dv_i}{dt} = f_\theta(v_i, a_i, \widehat{\text{msg}}_i, I_i), \quad \widehat{\text{msg}}_i = \sum_{j \in \mathcal{N}(i)} \widehat{W}_{ij}^2 g_\phi(v_j, a_j, v_i, a_i), \quad (2)$$

where a_i is a learned per-neuron embedding and $\widehat{W}_{ij}$ a learned per-edge weight.

Why the weight is squared. It puts the sign where (1) puts it. $\widehat{W}_{ij}^2 \geq 0$ makes the learned weight a conductance by construction, matching $W_{ij} \geq 0$, and leaves g_ϕ free to be negative — which it must be, since the factor it has to reproduce is $\text{relu}(v_j) (E_{ij} - v_i)$ and the driving force is negative on an inhibitory synapse. The alternative pairing, $\widehat{W}_{ij} g_\phi^2$, is the current generator’s convention, where the true weight is itself signed; used on conductance data it forbids g_ϕ from ever changing sign, so no inhibitory driving force can be represented and the polarity is pushed into $\widehat{W}_{ij}$ instead — against a W_{ij} that is non-negative by construction. Exactly one of the two may be squared. Throughout what follows, $\widehat{W}_{ij}^2$ is what the per-edge fit of §2.1 recovers, because it is what multiplies g_ϕ in the message; the raw parameter is its square root and is never compared with anything.

Only dv_i/dt is supervised. *Nothing in the loss observes $\widehat{\text{msg}}$* , and that is the whole difficulty: the message is an internal variable, so any rescaling of it that f_θ can undo leaves the training objective exactly unchanged. §2 first measures what the network computes; §2.2 makes that statement precise and undoes it.

2 The readout, as least squares

The question is an empirical one: does the trained network obey the generator’s equation (1) at all, and if so with which constants? The two are answered by the same fit. Impose the generator’s form on what the network computes, ask least squares for the constants that fit it best, and read the residual: the coefficients are the recovered parameters, and the R^2 is how well the hypothesis holds on that edge or that neuron. An edge whose message cannot be written as $W_{ij} \text{relu}(v_j) (E_{ij} - v_i)$ reports a low R^2 and its constants mean nothing; one that fits at 0.999 has a message the generator could have produced, and then the only question left is whether the constants are the right ones.

2.1 Per edge: W_{ij} and E_{ij}

Write $u_t = \text{relu}(v_j(t))$. The generator’s synaptic form, multiplied out, is

$$\text{msg}_{ij}(t) = W_{ij} u_t (E_{ij} - v_i(t)) + C_{ij} = \underbrace{(W_{ij} E_{ij})}_{b_1} u_t + \underbrace{(-W_{ij})}_{b_2} (u_t v_i(t)) + \underbrace{C_{ij}}_{b_3}, \quad (3)$$

linear in (b_1, b_2, b_3) . The target is the model’s own per-edge message, $\widehat{\text{msg}}_{ij}(t)$, taken from its forward pass. Stack the n usable frames $t_1 \dots t_n$ of this edge into a design matrix A and the model’s messages into y :

$$A = \begin{pmatrix} u_{t_1} & u_{t_1} v_i(t_1) & 1 \\ u_{t_2} & u_{t_2} v_i(t_2) & 1 \\ \vdots & \vdots & \vdots \\ u_{t_n} & u_{t_n} v_i(t_n) & 1 \end{pmatrix} \in \mathbb{R}^{n \times 3}, \quad b = \begin{pmatrix} b_1 \\ b_2 \\ b_3 \end{pmatrix}, \quad y = \begin{pmatrix} \widehat{\text{msg}}_{ij}(t_1) \\ \vdots \\ \widehat{\text{msg}}_{ij}(t_n) \end{pmatrix}. \quad (4)$$

The problem is $\min_b \|Ab - y\|_2^2$, whose normal equations $A^\top A b = A^\top y$ give

$$b = (A^\top A)^{-1} A^\top y, \quad W_{ij} = -b_2, \quad E_{ij} = -b_1/b_2, \quad C_{ij} = b_3. \quad (5)$$

$A^\top A$ is 3×3 whatever n is.

Which rows are usable. Three unknowns, so the fit requires $n_{\min} = 8$ usable rows per edge. A row constrains W_{ij} and E_{ij} only where the presynaptic cell is active; at $u_t = 0$ it is $[0, 0, 1]$ and fixes the offset alone. The activity floor is the median of the positive $\text{relu}(v_j)$, pooled over a 64-frame probe, and u_t must exceed it.

Choosing the frames. A quarter of this connectome’s edges have a presynaptic cell active in about 2% of the frames, which yields some five rows in a 256-frame draw. A uniform draw four times larger moves that share by about four points, and the sampler exhausts memory near $n = 4096$. Two passes instead:

1. A uniform draw of $n = 256$, then, for each presynaptic cell seen active fewer than eight times, extra frames drawn from that cell’s own active frames, capped at 2048.
2. 768 further frames, streamed in chunks of 256, their contributions added to the ten sums only for edges still under $n_{\min}$. $A^\top A$ and $A^\top y$ are sums over frames, so this adds to sums already formed and leaves every other edge untouched.

Together they fit 99.6% of the 434,112 edges.

Why E_{ij} is the fragile one. W_{ij} is a coefficient; E_{ij} is a ratio of two, so its relative error is $(\sigma_E/E)^2 \approx (\sigma_{b_1}/b_1)^2 + (\sigma_{b_2}/b_2)^2$ and diverges as $b_2 \rightarrow 0$. And b_2 is small not when the synapse is weak but when the postsynaptic voltage barely moves: if $v_i(t) \approx c$ over the usable frames then $u_t v_i(t) \approx c u_t$, the two columns are nearly the same vector, and only the combination $b_1 + c b_2$ is determined. The guard is a t -test on that coefficient, from the same normal equations:

$$\text{se}(b_2) = \sqrt{\hat{\sigma}^2 [(A^\top A)^{-1}]_{22}}, \quad t = \frac{|b_2|}{\text{se}(b_2)}, \quad E_{ij} \text{ kept only where } t \geq 3. \quad (6)$$

Everything below it is recorded as unidentified rather than dropped silently. On the conductance run used throughout: 432,517 of the 434,112 edges are fitted (99.6%), and of those, 405,503 clear the gate and report a reversal (93.4% of all edges).

2.2 Per neuron: the update fit, and the gauge it leaves free

f_θ is an MLP: there is no τ in it to read off, no resting potential, no message gain. What it is, on this data, is close to affine in its inputs — so fit it. Per neuron, taking $f(I)$ linear, regress the model’s own dv_i/dt on its own inputs:

$$\widehat{\frac{dv_i}{dt}} = \alpha_0 + \alpha_1 v_i + \alpha_2 \widehat{\text{msg}_i} + \alpha_3 I_i, \quad (7)$$

again $b = (A^\top A)^{-1} A^\top y$, now 4×4 per neuron and accumulated over frames so nothing of size $N \times n_{\text{frames}}$ is ever held. Greek, so the coefficients cannot be confused with the per-neuron embedding a_i of (2). The fit’s R^2 is reported: it is how far the model’s update is from affine in the message, and a low value invalidates all four coefficients together.

What the generator’s coefficients would be. Divide (1) by τ_i and it is already of exactly this form,

$$\frac{dv_i}{dt} = \frac{V_i^{\text{rest}}}{\tau_i} - \frac{1}{\tau_i} v_i + \frac{1}{\tau_i} \text{msg}_i + \frac{1}{\tau_i} I_i, \quad (8)$$

so a network obeying it exactly would return

$$\alpha_0 = \frac{V_i^{\text{rest}}}{\tau_i}, \quad \alpha_1 = -\frac{1}{\tau_i}, \quad \alpha_2 = +\frac{1}{\tau_i}, \quad \alpha_3 = \frac{1}{\tau_i}. \quad (9)$$

(9) holds in *one* gauge, and (10) below is what breaks it: that transformation moves α_0 and α_2 — precisely the two coefficients (9) pins — while leaving dv_i/dt untouched, so a network obeying the generator exactly returns (9) only if it happens to sit at $k_i = 1$, $\beta_i = 0$. What survives is $\alpha_1 = -1/\tau_i$, which (10) does not touch: the time constant is the one thing readable off the fit with no gauge argument at all.

The redundancy. Rescale and shift a neuron’s message, and absorb it in the two coefficients that multiply and offset it:

$$\widehat{\text{msg}_i} \mapsto \frac{1}{c} \widehat{\text{msg}_i} + d, \quad \alpha_2 \mapsto c \alpha_2, \quad \alpha_0 \mapsto \alpha_0 - c \alpha_2 d, \quad (10)$$

leaving dv_i/dt , hence the loss, unchanged for any $c \neq 0$ and any d : training cannot prefer one member of this two-parameter family over another.

Which member the trained model ended up as is then a question about *this* model, and the readout’s first job is to answer it. Suppose the model’s message is the true one seen through such a transformation:

$$\widehat{\text{msg}_i} = \frac{1}{k_i} \text{msg}_i + \beta_i. \quad (11)$$

That is (10) at $c = k_i$, $d = \beta_i$: different letters because (c, d) are free, while (k_i, β_i) are the particular pair relating this one model to the truth, per neuron, and unknown until measured. It is a supposition and it can be false — a message off the family is one whose per-edge form is not the generator’s at all, which is §2.3 — so it is tested by the per-edge R^2 rather than assumed. Either *fix* the gauge during training, where `coeff_f_theta_msg_gain` drives $k_i \rightarrow 1$ and `coeff_g_phi_silent` drives $\beta_i \rightarrow 0$, or *undo* it afterwards by measuring (k_i, β_i) . This document does the second.

Substituting (11) into the fit (7) puts the model’s update in terms of the *true* message,

$$\frac{dv_i}{dt} = (\alpha_0 + \alpha_2 \beta_i) + \alpha_1 v_i + \frac{\alpha_2}{k_i} \text{msg}_i + \alpha_3 I_i, \quad (12)$$

which is the generator’s own form (8). Both hold on every trajectory the network produces, so their coefficients must agree one by one, and matching them against (9) gives three identities:

$$\text{leak:} \quad \alpha_1 = -\frac{1}{\tau_i} \quad \implies \quad \tau_i = -1/\alpha_1, \quad (13)$$

$$\text{message:} \quad \frac{\alpha_2}{k_i} = \frac{1}{\tau_i} \quad \implies \quad k_i = \tau_i \alpha_2 = -\alpha_2/\alpha_1, \quad (14)$$

$$\text{constant:} \quad \alpha_0 + \alpha_2 \beta_i = \frac{V_i^{\text{rest}}}{\tau_i} \quad \implies \quad V_i^{\text{rest}} = \tau_i (\alpha_0 + \alpha_2 \beta_i). \quad (15)$$

(14) is worth reading twice: $k_i = 1$ exactly when $\alpha_2 = -\alpha_1$, which is the generator’s constraint from (9). The gauge is fixed precisely when the model’s two middle coefficients are equal and opposite — which is what `coeff_f_theta_msg_gain` charges during training, and it needs no τ to do it.

Applying them. The two fits return numbers about the model: per edge (b_1, b_2, b_3) and from them $W_{ij}^{\text{fit}}, E_{ij}, C_{ij}$; per neuron ($\alpha_0, \alpha_1, \alpha_2, \alpha_3$). The identities (13)–(15) are what turn those into the generator’s parameters, and they are used in this order:

$$(13) \quad \tau_i = -1/\alpha_1 \quad \text{the time constant, directly,} \quad (16)$$

$$(14) \quad k_i = -\alpha_2/\alpha_1 \quad \text{the message scale, which } W_{ij} \text{ is in units of,} \quad (17)$$

$$(15) \quad V_i^{\text{rest}} = \tau_i (\alpha_0 + \alpha_2 \beta_i) \quad \text{the resting level, offset included.} \quad (18)$$

So τ comes straight out of the update fit, W_{ij} needs the scale from (14) applied to the edge fit, and V_i^{rest} needs the offset that (15) says the update hid in it — the two corrections below. E_{ij} needs neither, for the reason given in §2.3; what it needs instead is a caveat, because the one error it is exposed to is the one no correction can remove.

W_{ij} : divide by the scale. By (14) the message is in units of $1/k_i$, and the fitted b_2 inherits that, so the weight in the generator’s units is

$$W_{ij}^{\text{corrected}} = k_i W_{ij}^{\text{fit}} = -\frac{\alpha_2}{\alpha_1} (-b_2), \quad (19)$$

both coefficients being the postsynaptic neuron’s own, from (7). Without this the same weights score $R^2 = -2.14$ where the corrected ones score 0.995 — the uncorrected number is kept as `Wij_R2_uncorrected` so the difference is visible rather than assumed.

V_i^{rest} : add the offset back. By (15) the level the update fit reads is short by exactly the neuron’s own incoming message offset:

$$V_i^{\text{rest, corrected}} = \tau_i \left[\underbrace{\alpha_0}_{\text{regression (7)}} + \underbrace{\alpha_2 \sum_j C_{ij}}_{C_{ij}=b_3 \text{ of (5)}} \right], \quad (20)$$

2.3 E_{ij} : why there is no correction, and what replaces one

The scale k_i divides out. The substitution, step by step. Start from the ansatz (11), which is a statement about neuron i ’s *total* incoming message:

$$\widehat{\text{msg}}_i = \frac{1}{k_i} \text{msg}_i + \beta_i \quad \text{with} \quad \text{msg}_i = \sum_j \text{msg}_{ij}, \quad \widehat{\text{msg}}_i = \sum_j \widehat{\text{msg}}_{ij}. \quad (11)$$

Step 1 — take it down to one edge. The fit of §2.1 works on a single synapse, so the ansatz has to be read per edge. Assume the model applies the same scale k_i to every edge into i and splits the offset among them,

$$\widehat{\text{msg}}_{ij}(t) = \frac{1}{k_i} \text{msg}_{ij}(t) + C_{ij}, \quad \sum_j C_{ij} = \beta_i. \quad (21)$$

k_i is a property of the receiving neuron, so carrying one scale across its edges is forced; splitting β_i into per-edge constants is not, and that assumption is exactly what the next paragraph removes.

Step 2 — put the generator’s synapse in. The true per-edge message is $\text{msg}_{ij}(t) = W_{ij} u_t (E_{ij} - v_i(t))$ with $u_t = \text{relu}(v_j(t))$, so

$$\widehat{\text{msg}}_{ij}(t) = \frac{1}{k_i} W_{ij} u_t (E_{ij} - v_i(t)) + C_{ij}. \quad (22)$$

Step 3 — multiply out the bracket. Distributing u_t over $E_{ij} - v_i(t)$ separates the two things that multiply it:

$$\widehat{\text{msg}}_{ij}(t) = \frac{W_{ij} E_{ij}}{k_i} u_t - \frac{W_{ij}}{k_i} (u_t v_i(t)) + C_{ij}. \quad (23)$$

Step 4 — read off the coefficients. The fit (3) regresses $\widehat{\text{msg}}_{ij}$ on the three columns u_t , $u_t v_i(t)$ and 1. Equation (23) is already in that form, so its three coefficients are b_1, b_2, b_3 :

$$b_1 = \frac{W_{ij} E_{ij}}{k_i}, \quad b_2 = -\frac{W_{ij}}{k_i}, \quad b_3 = C_{ij}. \quad (24)$$

Step 5 — divide. Both b_1 and b_2 carry the same factor $1/k_i$, and (5) takes their ratio:

$$-\frac{b_1}{b_2} = -\frac{W_{ij} E_{ij}/k_i}{-W_{ij}/k_i} = \frac{W_{ij} E_{ij}}{W_{ij}} \cdot \frac{k_i}{k_i} = E_{ij} \text{ exactly, for any } k_i. \quad (25)$$

W_{ij} is a single coefficient and keeps the $1/k_i$, which is why (19) multiplies it back; E_{ij} is a *ratio* of two coefficients that carry the same factor, so it divides out. The offset cannot reach E_{ij} either — C_{ij} is b_3 , and b_3 appears in neither b_1 nor b_2 . That is the precise content of “ E_{ij} needs neither correction”.

But E_{ij} is exposed to a contamination W_{ij} is not. Step 1 above is where the whole result rests, and it is an assumption. The ansatz (11) is a statement about neuron i ’s *total* message; writing it per edge as (21) adds the claim that whatever the model does beyond rescaling is a per-edge *constant*. Nothing in the training objective requires that. Relax it in the one direction that matters — let each edge also carry a component proportional to the drive, with no driving force in it:

$$\widehat{\text{msg}}_{ij}(t) = \frac{1}{k_i} W_{ij} u_t (E_{ij} - v_i(t)) + C_{ij} + D_{ij} u_t. \quad (26)$$

$D_{ij} u_t$ is exactly the *current* family’s form, $W \text{relu}(v_j)$, and it is collinear with the $W_{ij} E_{ij}$ term: both live in the first column of A . The fit therefore returns

$$b_1 = \frac{W_{ij} E_{ij}}{k_i} + D_{ij}, \quad b_2 = -\frac{W_{ij}}{k_i}, \quad \implies \quad W_{ij}^{\text{corrected}} = W_{ij} \text{ (untouched)}, \quad E_{ij}^{\text{fit}} = E_{ij} + \frac{k_i D_{ij}}{W_{ij}}. \quad (27)$$

So a current-form component leaves the conductance *exactly* right and biases the reversal, by an amount that grows as W_{ij} shrinks. This is not a gauge and there is no correction for it: b_1 is one column’s coefficient and cannot be split into $W_{ij} E_{ij}$ and $k_i D_{ij}$ from a single edge’s message. The two are unidentifiable together, and any procedure claiming to separate them is using an assumption from outside the data.

What is reported instead. Two numbers, which separate the two ways E_{ij} can be wrong. The *variance* failure — b_2 small because v_i barely moved, so its reciprocal explodes — is handled by the t -gate (6) and quantified by the delta-method standard error of the ratio, propagated through both coefficients off the same $(A^\top A)^{-1}$:

$$\text{var}(E_{ij}) = \frac{\hat{\sigma}^2}{b_2^2} \left[V_{11} + \left(\frac{b_1}{b_2} \right)^2 V_{22} - 2 \frac{b_1}{b_2} V_{12} \right], \quad V = (A^\top A)^{-1}. \quad (28)$$

The *bias* failure of (27) is invisible to that: it shifts the estimate without widening it. It shows up only as an observed error far larger than (28) predicts, which is what the ratio $\text{median}|E_{ij}^{\text{fit}} - E_{ij}| / \text{medianse}(E_{ij})$ measures. Near 1, the reversal is recovered as well as the data allows and more frames would help. Far above 1, the message carries a term the conductance form has no column for, and more frames will not move it. Both are reported per run as `Eij_se_median` and `Eij_err_over_se`, beside the two-form comparison (`current_form_r2_median` against `conductance_form_r2_median`) that says how much the driving-force column was worth in the first place.

A consequence worth stating plainly. W_{ij} and E_{ij} fail independently and for opposite reasons. W_{ij} is immune to the contamination (26) but carries the gauge parameter k_i , so it is wrong by one global factor whenever the message scale is unpinned. E_{ij} is immune to k_i but not to the contamination. A run reporting a good E_{ij} and a W_{ij} off by a uniform factor has an unfixed-gauge problem; one reporting a good W_{ij} and a scattered E_{ij} has a message whose form is not purely conductance. The two diagnoses are not interchangeable, and neither is read off the other’s number.

3 A worked example

Figure 1 shows the readout applied to neuron 2895 of a conductance network trained at process noise $\sigma = 0.05$ on the flyvis connectome ($N = 13,741$ neurons, 434,112 edges).

The V^{rest} identity closes. Equation (15) predicts $V^{\text{rest}} = \tau\alpha_0 + \tau\alpha_2\beta$, and the second term is the fitted offset of panel (c). Substituting: $-0.083 + 2.334 = +2.251$ against the generator’s $+2.910$ — 77% of the resting potential recovered, of which essentially all comes from the message’s offset and almost none from $\tau\alpha_0 = -0.083$. Reading $\tau\alpha_0$ alone as V^{rest} , which is what dropping the $\alpha_2\beta$ term amounts to, would be wrong by 3 V.

The per-edge offsets sum to the per-neuron one. Panel (c) reports $+2.334$ measured on the total message against $+1.510$ from summing the eight fitted C_{ij} ; the gap is the three synapses the model pruned, which have no fit to contribute one. The two stages agree on a quantity neither of them imposes.

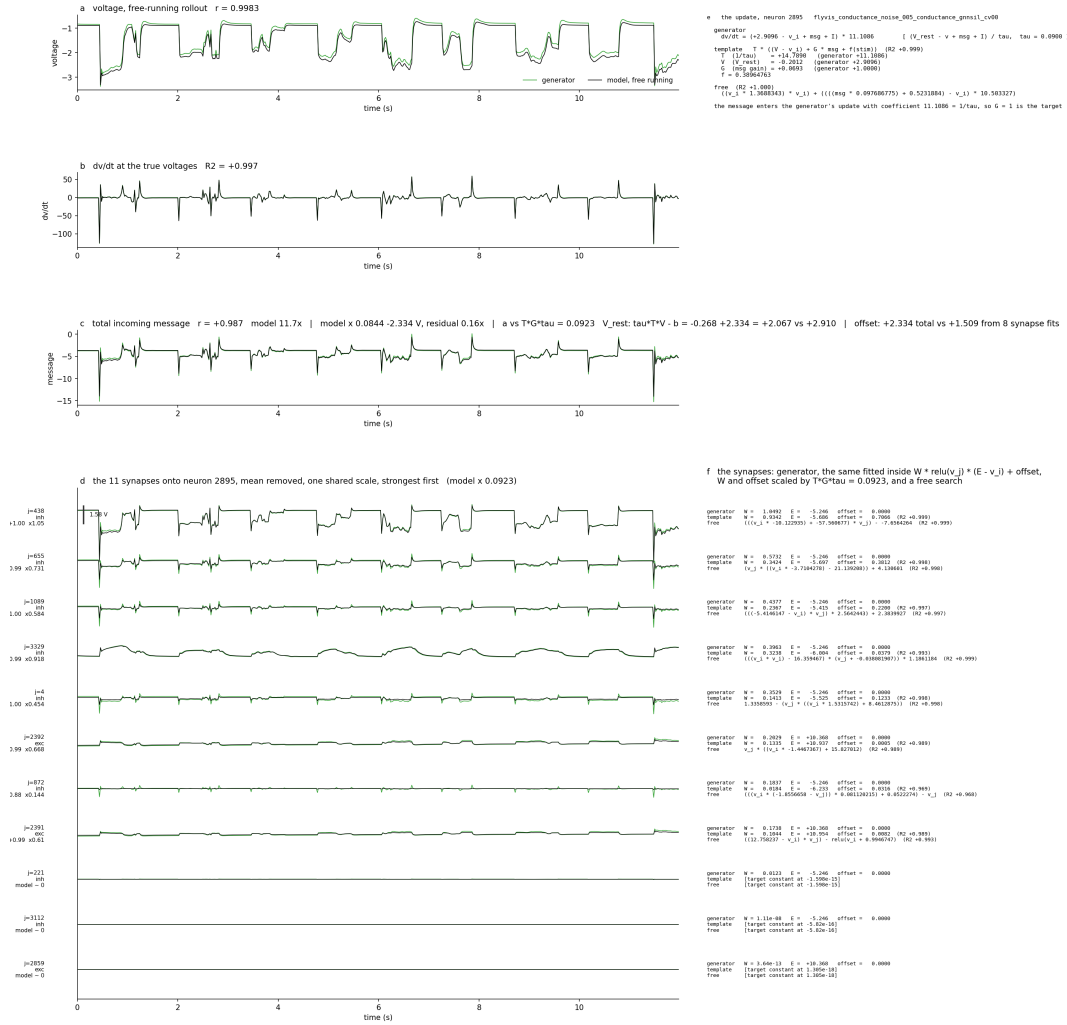


Figure 1: One neuron, term by term. **(a)** free-running rollout, generator in green and model in black, $r = 0.998$. **(b)** dv/dt at the true voltages, $R^2 = 0.997$. **(c)** the total incoming message: the learned one after the affine correction of (11), against the generator's. The heading reports the fitted scale 0.0844 against $k = -\alpha_2/\alpha_1 = 0.0923$ from panel (e) — two independent routes to the same number, agreeing to 9% — and the offset $b = -2.334$ V. **(d)** each of the 11 synapses separately, mean removed, on one shared voltage scale so that a synapse the model dropped looks flat rather than being magnified to full height. **(e)** the update fit of (7): $-\alpha_1 = 14.79$ against the generator's $1/\tau = 11.11$, and $k = -\alpha_2/\alpha_1 = 0.0693$ against a target of 1. **(f)** per synapse, the generator's ($W, E, \text{offset} = 0$) against the template fit of (5), and a free symbolic search for comparison.